

Déploiement et installation

La traçabilité agricole, de la parcelle au marché.

Version du 2026-10-24

Déploiement

> **État** : les Dockerfiles et `docker-compose.prod.yml` n'ont **pas pu être construits dans l'environnement de développement** (pas de démon Docker). Ils sont construits par la CI (job `docker`). Faire un premier déploiement sur un serveur de préproduction avant toute mise en production.

Prérequis

Un serveur Linux avec Docker, un nom de domaine, et un reverse proxy HTTPS devant le portail (Caddy, Traefik, nginx ou équilibreur du cloud). Le compose n'expose que le portail en HTTP (port 8080) ; l'API n'est pas publiée, le portail la relaie sur `/v1`.

Première mise en service

1. `cp .env.example .env` et remplacer **toutes** les valeurs (mots de passe longs et aléatoires, `JWT_SECRET` ≥ 48 caractères). Ne jamais versionner `.env`.
2. `docker compose -f docker-compose.prod.yml up -d --build`
   - `db` crée le rôle applicatif `agritrace_app` (non propriétaire, non superutilisateur) au premier démarrage ;
   - `migrate` applique les migrations avec le propriétaire puis s'arrête ;
   - `api` démarre ensuite avec le rôle applicatif (la sécurité par ligne dépend de ce point : ne jamais faire tourner l'API avec le propriétaire).
3. Créer l'administrateur de plateforme : `docker compose -f docker-compose.prod.yml run --rm -e DATABASE_URL=postgres://agritrace_owner:<mdp>@db:5432/agritrace -e ADMIN_PASSWORD=<mdp-admin> api node scripts/create-platform-admin.js admin@exemple.org "Nom"`
4. Ne **jamais** lancer `seed-demo.js` en production.

Mises à jour

`git pull && docker compose -f docker-compose.prod.yml up -d --build` : les migrations s'appliquent avant le redémarrage de l'API. Les migrations sont additives ; sauvegarder avant chaque mise à jour.

Réglages de l'API

Variable	Rôle
`DATABASE_URL`, `JWT_SECRET`	obligatoires. En production (`NODE_ENV=production`) : `JWT_SECRET` de 32 caractères minimum, non prévisible (`openssl rand -base64 48`), sinon l'API refuse de démarrer

Variable	Rôle
`*_FILE`	chaque secret peut être fourni par fichier (Docker/Kubernetes secrets) : `JWT_SECRET_FILE`, `DATABASE_URL_FILE`, `METRICS_TOKEN_FILE`, `SECRETS_KEY_FILE`, `DOCUMENTS_KEY_FILE`, `SMTP_URL_FILE`, `BOOTSTRAP_TOKEN_FILE`
`DOCUMENTS_KEY`	**chiffre les documents au repos** (AES-256-GCM) : `openssl rand -base64 32`. À sauvegarder séparément : sans elle les documents chiffrés sont illisibles. Les documents anciens restent lisibles ; les nouveaux sont chiffrés
`SECRETS_KEY`	chiffre les secrets de double authentification : `openssl rand -base64 32`, distincte de `JWT_SECRET` (changer `JWT_SECRET` après un incident ne doit pas casser la 2FA)
`REQUIRE_2FA`	`true` : les titulaires de droits sensibles (équipe, finances, audit, validations) et les administrateurs de plateforme doivent activer la double authentification pour continuer
`LOGIN_THROTTLE_LIMIT`	connexions par minute et par adresse (10 par défaut ; ne pas relever en production)
`ENABLE_API_DOCS`	`true` réactive la documentation Swagger (fermée en production par défaut)
`BOOTSTRAP_TOKEN`	serveur local vierge : l'initialisation est acceptée depuis ce poste, ou depuis le réseau avec l'en-tête `X-Bootstrap-Token`
`CLAMAV_HOST`, `CLAMAV_PORT`, `CLAMAV_REQUIRED`	analyse antivirus (ClamAV, clamd) des documents déposés ; `CLAMAV_REQUIRED=true` refuse le dépôt si l'analyseur est injoignable
`DOCUMENTS_DIR`	dossier des pièces (volume `documents`)
`CORS_ORIGINS`	origines autorisées si le portail est sur un autre domaine
`TRUST_PROXY`	nombre de proxys de confiance (2 dans le compose : proxy HTTPS + nginx) ; vérifier ce nombre, sinon l'IP du journal d'audit et du limiteur est fausse
`THROTTLE_LIMIT`	requêtes/minute/IP (120 par défaut)
`METRICS_TOKEN`	active `/v1/metrics` (404 sinon)
`LOG_REQUESTS`	journal d'accès JSON : méthode, route normalisée, statut, durée ; jamais de corps ni de jeton

Points de santé

`GET /v1/health` (vivacité) et `GET /v1/ready` (base joignable). L'arrêt est propre (`SIGTERM` : requêtes en cours terminées, pool fermé).

PostGIS

Les fichiers `docker-compose*.yml` utilisent l'image `postgis/postgis:16-3.4` (PostgreSQL 16 + PostGIS). `node scripts/migrate.js` active l'extension et la colonne géométrique `parcels.geom` quand elle est disponible, et indique `PostGIS : actif` ou `PostGIS : inactif` dans sa sortie. Avec un PostgreSQL sans PostGIS, tout fonctionne : le calcul des chevauchements revient à l'application (voir `docs/ECARTS_CDC.md`). Pour une base existante dont l'utilisateur n'est pas super-utilisateur, faire exécuter une fois `CREATE EXTENSION postgis;` par un super-utilisateur, puis relancer `migrate`.